머신러닝

4주차. 평가(Evaluation)

학습 목표

- 정확도(Accuracy)
- 오차행렬(Confusion Matrix)
- 정밀도(Precision)
- 재현율(Recall)
- F1 스코어
- ROC AUC

Scikit-Learn 사용 방법 및 순서

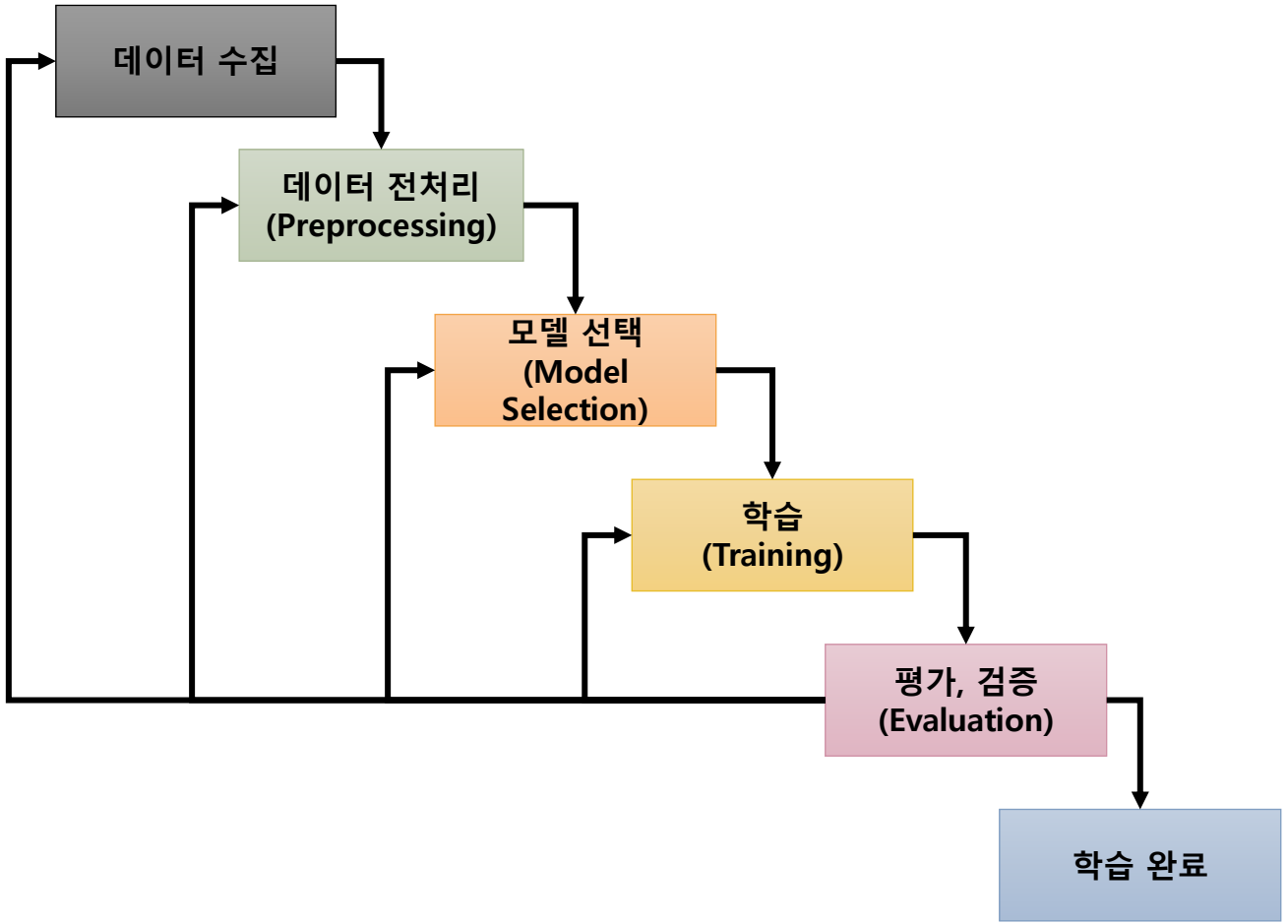
- **데이터 수집**
  - 해결하고자 하는 문제를 정의하고
  - 관련 된 데이터 수집(공개 데이터, 웹 크롤링, 로그 데이터 수집 등)
  - 수집된 데이터(10) = 학습 데이터(7) + 검증 데이터(3)

- **데이터 전처리**
  - 수집된 데이터를 학습에 적용하기 위한 형태로 변형
  - 불필요한 요소 또는 중복 제거, 데이터 누락 여부 확인 등
  - 데이터 정제라고도 부름

- **모델 선택**
  - 학습 방법과 알고리즘 선택
  - 데이터의 성격에 따라 지도학습, 비지도학습, 강화학습 모델

- **학습**
  - 주어진 데이터들을 기반으로 선택한 모델로 학습
  - 학습 데이터(Training Data) 사용

- **평가 및 검증**
  - 검증 데이터(Test Data)사용
  - 학습 결과를 평가하여 성능 검증
  - 결과에 따라 모델의 알고리즘 또는 조건을 변경하고 다시 학습

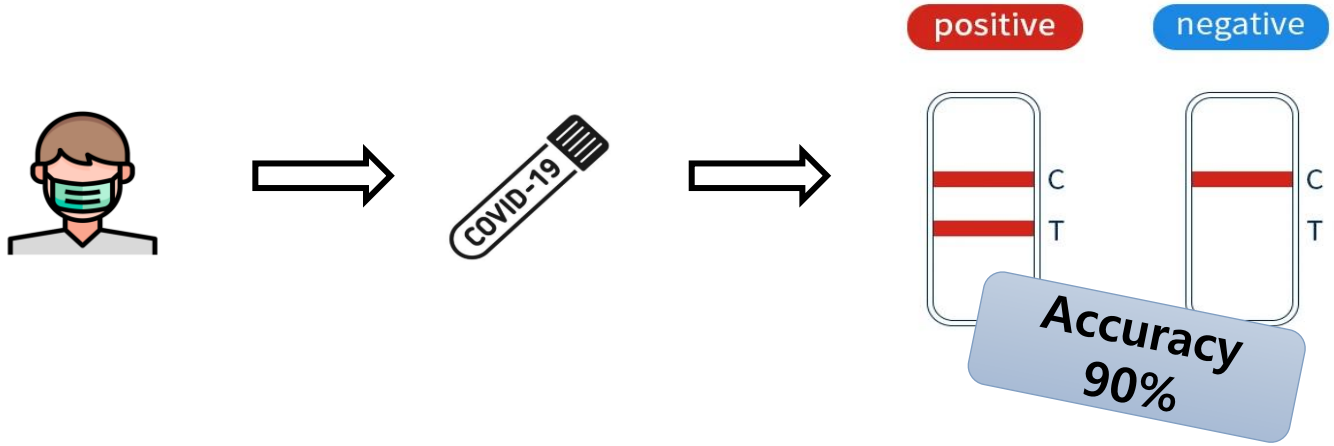


정확도와 오차행렬

- **정확도(Accuracy)의 의미**

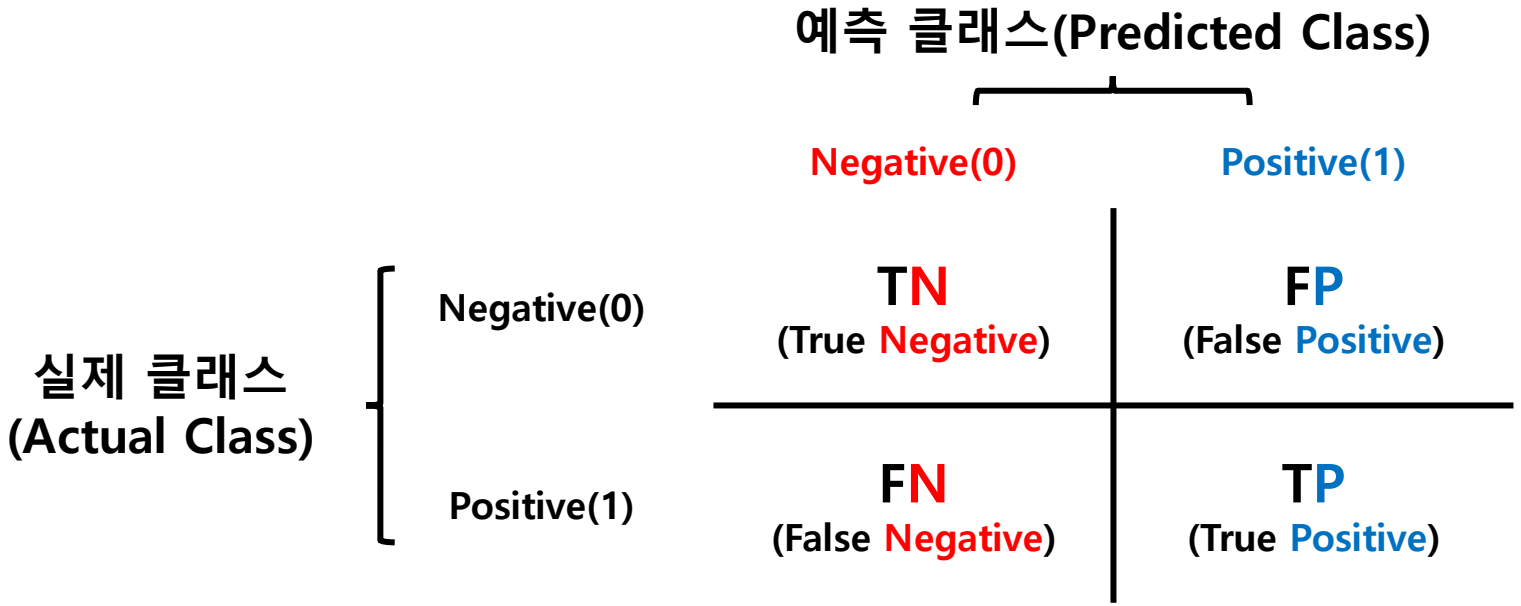
$$\text{정확도}(\textbf{Accuarcy}) = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

- 모델의 성능을 평가하는 대표적인 지표 중 하나
- 전체 데이터 중에서 모델이 맞게 예측한 비율



정확도와 오차행렬(계속)

- **오차 행렬(Confusion Matrix)**
  - 분류 모델의 성능을 보다 자세히 분석할 수 있는 평가 지표
  - 단순한 정확도만으로는 알 수 없는 예측 결과의 세부구조를 보여줌
  - 예측 오류가 얼마인지와 어떠한 유형의 예측 오류가 발생하고 있는지를 보여줌



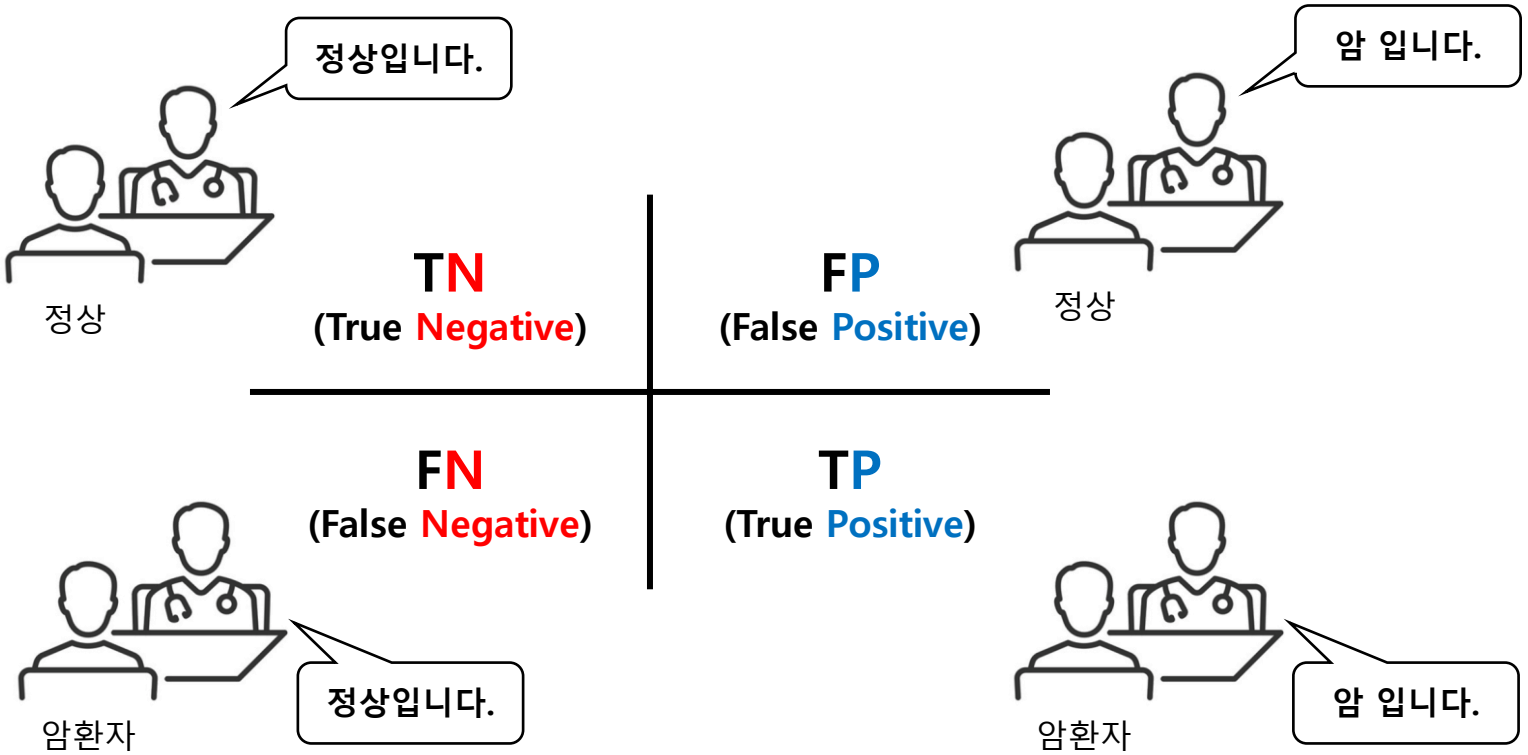
TN= 예측: 0, 실제: 0
TP= 예측: 1, 실제: 1
FN= 예측: 0, 실제: 1
FP= 예측: 1, 실제: 0

정확도와 오차행렬(계속)

- **실제 클래스와 예측의 관계**
  - Ture: 클래스 예측이 잘됨(TN, TP)
  - False: 클래스 예측이 잘못됨(FN, FP)

Class

Negative(0): 정상
Positive(1): 암환자



정확도와 오차행렬(계속)

- 오차행렬에서의 정확도(Accuracy) 분석

TN (True Negative)	**FP** (False Positive)
FN (False Negative)	**TP** (True Positive)



정상 (990)



암환자 (10)

TN(990) (정상환자 예측 성공) (0 -> 0)	**FP(0)** (예측실패) (정상을 암환자로 예측) (0 -> 1)
FN(10) (예측실패) (암환자를 정상으로 예측) (1 -> 0)	**TP(0)** (암환자 예측성공) (1 -> 1)

정확도 (Accuarcy)

$$\frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}} = \frac{(TN + TP)}{(TN + FP + FN + TP)}$$

정확도 계산

$$\frac{(990 + 0)}{(990 + 0 + 10 + 0)} = \frac{(990)}{(1000)} = 0.99\%$$

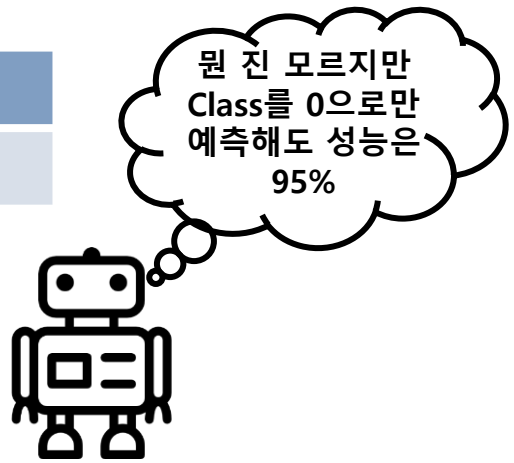
정확도 수치로는 성능이 좋아 보이지만 암환자를 예측하지 못함

정확도와 오차행렬(계속)

- **정확도 지표의 장점**
  - 직관적이고 이해하기 쉬움
  - 균형잡힌 클래스 분포에 유용
    - 클래스 간의 분포가 고르게 있을 때 정확도는 효과적인 평가 지표

- **정확도 지표의 단점**
  - 불균형 데이터에 취약
    - 클래스의 비율이 치우치면 모델 성능이 왜곡됨
  - 오탐/누락에 대한 정보 부족
    - 어떤 종류의 오류가 발생했는지를 보여주지 않음으로 세밀한 평가가 어려움
  - 위험한 판단 유도
    - 모델 성능이 좋아 보이지만 실제로는 아무런 의미가 없음

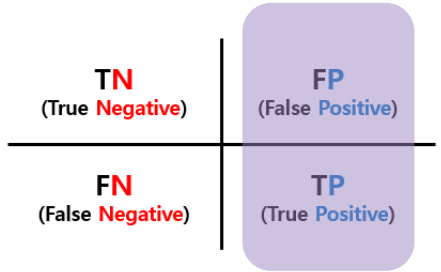
Class 0	Class 1
95개	5개



정밀도와 재현율

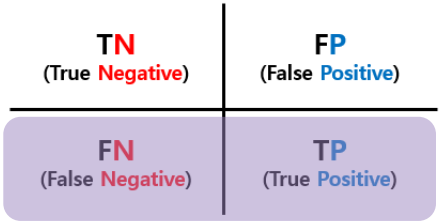
- **정밀도(Precision)**
  - 예측을 Positive로 한 대상 중 예측과 실제 값이 Positive로 일치한 데이터의 비율
  - **"양성이라고 예측한 것들$(TP + FP)$ 중 진짜 양성 (TP) 이 몇 % 인가?"**
  - FP(거짓정보) **값이 줄수록** 정밀도 증가

$$Precision = \frac{TP}{TP + FP}$$



- **재현율(Recall) / 민감도(Sensitivity)**
  - 실제 값이 Positive인 대상 중에 예측과 실제 값이 Positive로 일치한 데이터의 비율
  - **"실제 값$(TP + FN)$ 중에서 양성 (TP) 을 찾은 것이 몇 % 인가?"**
  - FP(실제 값 중 놓친거) **값이 줄수록** 재현율 증가

$$Recall = \frac{TP}{TP + FN}$$



정밀도와 재현율(계속)

- 정밀도와 재현율 평가



정상
(990)



암환자
(10)

TN(990) (정상환자 예측 성공) (0 -> 0)	**FP(0)** (예측실패) (정상을 암환자로 예측)) (0 -> 1)
FN(10) (예측실패) (암환자를 정상으로 예측) (1 -> 0)	**TP(0)** (암환자 예측성공) (1 -> 1)

$$\text{accuracy} = \frac{990}{1000} = 0.99\%$$

$$Precision = \frac{TP}{TP + FP} = \frac{0}{0} = 0\%$$

$$Recall = \frac{TP}{TP + FN} = \frac{0}{10} = 0\%$$



정상
(975)



암환자
(25)

TN(970) (정상환자 예측 성공) (0 -> 0)	**FP(5)** (예측실패) (정상을 암환자로 예측)) (0 -> 1)
FN(10) (예측실패) (암환자를 정상으로 예측) (1 -> 0)	**TP(15)** (암환자 예측성공) (1 -> 1)

$$\text{accuracy} = \frac{985}{1000} = 0.985\%$$

$$Precision = \frac{TP}{TP + FP} = \frac{15}{20} = 0.75\%$$

$$Recall = \frac{TP}{TP + FN} = \frac{15}{25} = 0.6\%$$

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현

- **오차행렬 호출**

```
from sklearn.metrics import confusion_matrix
```

- **오차행렬 호출 입력 값**
  - y_true(필수): 정답(target/class) 데이터
  - y_pred(필수): 예측 데이터
  - labels: target 데이터 레이블

```
confusion_matrix(
    y_true,
    y_pred,
    *,
    labels=None,
    sample_weight=None,
    normalize=None,
)
```

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현

- **평가 모듈 호출**
  - accuracy_score()
  - precision_score()
  - recall_score()

```
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
```

```
accuracy_score(y_true, y_pred, *, normalize=True, sample_weight=None)
```

- **호출 입력 값**
  - y_true(필수): 정답(target/class) 데이터
  - y_pred(필수): 예측 데이터
  - average: 평균값의 형태 지정
    - 이진분류: binary(default)
    - 다중분류: None, micro, macro, weighted

```
precision_score(
    y_true,
    y_pred,
    *,
    labels=None,
    pos_label=1,
    average='binary',
    sample_weight=None,
    zero_division='warn',
)
```

```
recall_score(
    y_true,
    y_pred,
    *,
    labels=None,
    pos_label=1,
    average='binary',
    sample_weight=None,
    zero_division='warn',
)
```

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현



정상
(975)

TN(970)
(정상환자 예측 성공)
(0 -> 0)

FP(5)
(예측실패)
(정상을 암환자로 예측))
(0 -> 1)



암환자
(25)

FN(10)
(예측실패)
(암환자를 정상으로 예측)
(1 -> 0)

TP(15)
(암환자 예측성공)
(1 -> 1)

```
list1 = [0 for x in range(975)]              #정상인 0
list2 = [1 for x in range(25)]               #암환자 1

list_label = list1 + list2                   #정답 데이터(정상인 975명, 암환자 25명)

list3 = [0 for x in range(970)]              #TN = 정상인(0)을 정상인(0)으로 예측
list4 = [1,1,1,1,1]                          #FP = 정상인(0)을 암환자(1)로 예측
list5 = [0,0,0,0,0,0,0,0,0,0]                #FN = 암환자(1)를 정상인(0)으로 예측
list6 = [1 for x in range(15)]               #TP = 암환자(1)를 암환자(1)로 예측

list_pred = list3 + list4 + list5 + list6    #예측 결과(TN + FP + FN + TP)

print(list_label[970 :])
print(list_pred[970 :])
```

```
[0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

FP(5) FN(10) TP(15)

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현

```python
import numpy as np
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score

y_test = np.array(list_label)        # 정답 데이터(target)
y_pred = np.array(list_pred)         # 예측 데이터(predict)

print("오차 행렬")
print(confusion_matrix(y_test, y_pred))

print("\n==== 정확도, 정밀도, 재현율 ====")
print("정확도: ", accuracy_score(y_test, y_pred))
print("정밀도: ", precision_score(y_test, y_pred))
print("재현율: ", recall_score(y_test, y_pred))
```




```
오차 행렬
[[970   5]
 [ 10  15]]

==== 정확도, 정밀도, 재현율 ====

정확도:  0.985
정밀도:  0.75
재현율:  0.6
```

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현: 붓꽃데이터 다중분류 평가

- **붓꽃데이터 다중 분류 평가**

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

#붓꽃 데이터 로드
iris = load_iris()
iris_data = iris.data            #특성 데이터
iris_label = iris.target         #타겟 데이터

#학습/테스트 데이터 분류
X_train, X_test, y_train, y_test = train_test_split(iris_data, iris_label,
                                                    test_size = 0.2, random_state = 11)

#학습 모델 객체
dt_clf = DecisionTreeClassifier(random_state = 11)

dt_clf.fit(X_train, y_train)          #분류 학습
pred = dt_clf.predict(X_test)         #분류 예측
```

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현: 붓꽃데이터 다중분류 평가

- **예측 정확도 평가 방식**
  - 학습된 모델로 **예측한 결과 값(pred)**과 **테스트용 타겟 데이터(정답데이터: y_test)**와 비교

```
1 pred
```

```
array([2, 2, 1, 1, 2, 0, 1, 0, 0, 1, 1, 1, 1, 2, 2, 0, 2, 1, 2, 2, 1, 0,
       0, 1, 0, 0, 2, 1, 0, 1])
```

예측 데이터 (pred)

```
1 y_test
```

```
array([2, 2, 2, 1, 2, 0, 1, 0, 0, 1, 2, 1, 1, 2, 2, 0, 2, 1, 2, 2, 1, 0,
       0, 1, 0, 0, 2, 1, 0, 1])
```

정답 데이터 (y_test)

오차행렬 / 정밀도 / 재현율
- Scikit-learn 구현: 붓꽃데이터 다중분류 평가

```python
#평가모듈 가져오기
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score

#오차행렬
print("오차행렬")
print(confusion_matrix(y_test, pred))
print()

#정확도
print('예측 정확도: {0: .4f}'.format(accuracy_score(y_test, pred)))
print()

#정밀도
print('1 정밀도: ', precision_score(y_test, pred, average = None))
print('2 정밀도: ', precision_score(y_test, pred, average = "micro"))
print('3 정밀도: ', precision_score(y_test, pred, average = "macro"))
print('4 정밀도: ', precision_score(y_test, pred, average = "weighted"))
print()

#재현률
print('1 재현률: ', recall_score(y_test, pred, average = None))
print('2 재현률: ', recall_score(y_test, pred, average = "micro"))
print('3 재현률: ', recall_score(y_test, pred, average = "macro"))
print('4 재현률: ', recall_score(y_test, pred, average = "weighted"))
```

```
오차행렬
[[ 9  0  0]
 [ 0 10  0]
 [ 0  2  9]]

예측 정확도:  0.9333

1 정밀도:  [1.         0.83333333 1.        ]
2 정밀도:  0.9333333333333333
3 정밀도:  0.9444444444444445
4 정밀도:  0.9444444444444445

1 재현률:  [1.         1.         0.81818182]
2 재현률:  0.9333333333333333
3 재현률:  0.9393939393939394
4 재현률:  0.9333333333333333
```

정밀도 / 재현율 트레이드 오프

- **정밀도가 상대적으로 더 중요한 지표인 경우**
  - 실제 음성(Negative)인 데이터 예측을 양성(Positive)로 잘못 판단되는 경우
  - 잘못 잡아내는 것이 더 위험하거나 비용이 큰 경우

  ex) 추천 시스템, 광고 타겟팅, 스팸메일

찾아낸 것 중에서 얼마나 맞는 것이 많은가?

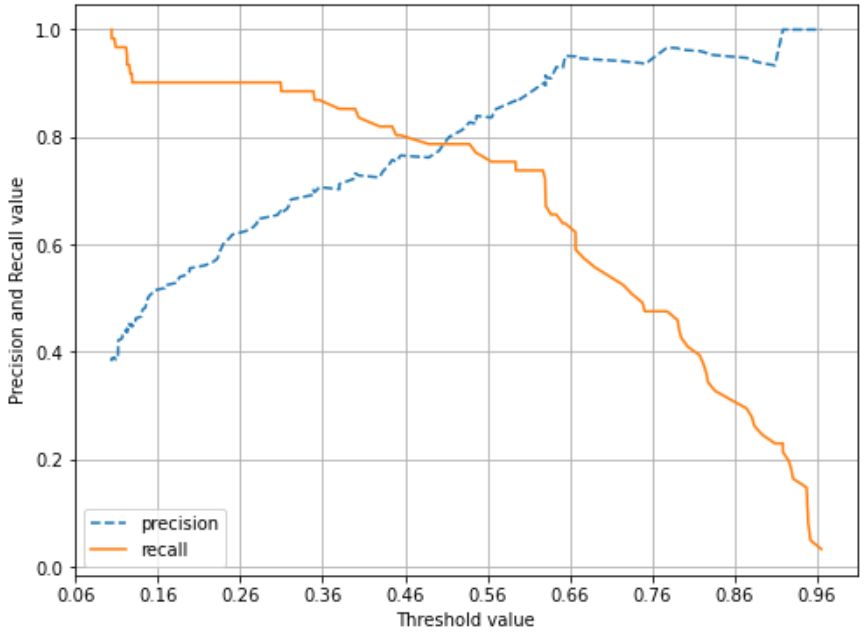
- **재현율이 상대적으로 더 중요한 지표인 경우**
  - 실제 양성(Positive)인 데이터 예측을 음성(Negative)으로 잘못 판단되는 경우
  - 놓치는 것이 더 위험한 경우로 놓쳤을 때 발생되는 비용이 큰 업무

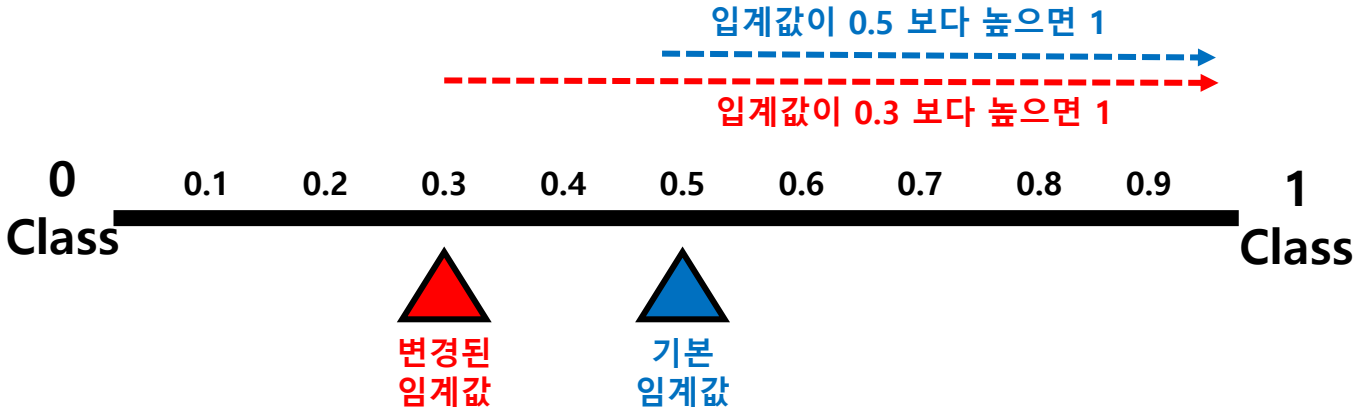
  ex) 의료 진단(암 등), 금융 사기 판별, 구조 요청 분석

실제로 맞는 것을 얼마나 놓치지 않고 찾아내는가?

정밀도 / 재현율 트레이드 오프

- **분류 결정 임곗값(Threshold)**
  - 분류하려는 업무의 특성상 정밀도 또는 재현율이 특별히 강조돼야 할 경우 결정 임곗값을 조정
  - 분류 결정 임계값이 낮을수록 positive로 예측할 확률이 높아짐(재현율 증가)





Threshold	Probability	Class
0.5	0.4	0
0.3		1

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 데이터 준비

- 데이터 전처리 함수 생성

```python
from sklearn.preprocessing import LabelEncoder

# Null 처리 함수
def fillna(df):
    df['Age'].fillna(df['Age'].mean(), inplace=True)
    df['Cabin'].fillna('N', inplace=True)
    df['Embarked'].fillna('N', inplace=True)
    df['Fare'].fillna(0, inplace=True)
    return df

# 머신러닝 알고리즘에 불필요한 피처 제거
def drop_features(df):
    df.drop(['PassengerId', 'Name', 'Ticket'], axis=1, inplace=True)
    return df


# 레이블 인코딩 수행.
def format_features(df):
    df['Cabin'] = df['Cabin'].str[:1]
    features = ['Cabin', 'Sex', 'Embarked']
    for feature in features:
        le = LabelEncoder()
        le = le.fit(df[feature])
        df[feature] = le.transform(df[feature])
    return df
```

```python
# 앞에서 설정한 데이터 전처리 함수 호출
def transform_features(df):
    df = fillna(df)
    df = drop_features(df)
    df = format_features(df)
    return df
```

fillna() 함수호출: 결측치 대체
drop_features(): 학습에 사용되지 않는 컬럼 삭제
format_features(): object(문자열)자료형, 수치형으로 인코딩

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 데이터 준비(계속)

- 피처 데이터 / 타겟(레이블) 데이터 분리

```
# 원본 데이터에 feature데이터 셋과 Label 데이터 셋 추출.
titanic_df = pd.read_csv('./titanic_train.csv')

y_titanic_df = titanic_df['Survived']

X_titanic_df= titanic_df.drop('Survived',axis=1)

X_titanic_df = transform_features(X_titanic_df)
```

원본 데이터 가져오기

'Survived' 컬럼만 빼서 타겟 데이터(y_titanic_df)로 분리

원본 데이터에서 'Survived' 컬럼만 삭제해서 피처 데이터(X_titanic_df)로 만들기

전처리 함수를 통해 피처 데이터 전처리

피처 데이터(X_titanic_df)

	Pclass	Sex	Age	SibSp	Parch	Fare	Cabin	Embarked
0	3	1	22.0	1	0	7.2500	7	3
1	1	0	38.0	1	0	71.2833	2	0
2	3	0	26.0	0	0	7.9250	7	3

타겟 데이터(y_titanic_df)

```
0    0
1    1
2    1
Name: Survived, dtype: int64
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 데이터 준비(계속)

- 학습 / 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X_titanic_df, y_titanic_df, \
                                                  test_size=0.2, random_state=11)
```

- 학습 알고리즘 import 및 객체 생성

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 결정트리, Random Forest, 로지스틱 회귀를 위한 사이킷런 Classifier 클래스 생성
dt_clf = DecisionTreeClassifier(random_state=11)
rf_clf = RandomForestClassifier(random_state=11)
lr_clf = LogisticRegression(solver='liblinear')
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 데이터 준비(계속)

- **학습 및 평가**

```
# DecisionTreeClassifier 학습/예측/평가
dt_clf.fit(X_train , y_train)
dt_pred = dt_clf.predict(X_test)
print('DecisionTreeClassifier 정확도: {0:.4f}'.format(accuracy_score(y_test, dt_pred)))

# RandomForestClassifier 학습/예측/평가
rf_clf.fit(X_train , y_train)
rf_pred = rf_clf.predict(X_test)
print('RandomForestClassifier 정확도:{0:.4f}'.format(accuracy_score(y_test, rf_pred)))

# LogisticRegression 학습/예측/평가
lr_clf.fit(X_train , y_train)
lr_pred = lr_clf.predict(X_test)
print('LogisticRegression 정확도: {0:.4f}'.format(accuracy_score(y_test, lr_pred)))
```

```
DecisionTreeClassifier 정확도: 0.7877
RandomForestClassifier 정확도:0.8547
LogisticRegression 정확도: 0.8659
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: predict_proba()

- **predict_proba()**
  - 분류 알고리즘에서 학습된 모델에서 각각의 클래스에 해당되는 학률 반환
  - 높은 확률로 반환된 값으로 분류

```
pred_proba = lr_clf.predict_proba(X_test)
pred  = lr_clf.predict(X_test)

print('pred_proba()결과 Shape : {0}'.format(pred_proba.shape))
print('pred_proba array에서 앞 3개만 샘플로 추출 \n', pred_proba[:3], pred[:3])
```



```
pred_proba()결과 Shape : (179, 2)
pred_proba array에서 앞 3개만 샘플로 추출
 [[0.44935225 0.55064775]
 [0.86335511 0.13664489]
 [0.86429643 0.13570357]] [1 0 0]
```

lr_clf.predict_proba(x_test)

0	1
[0.44935225	0.55064775]
[0.86335511	0.13664489]
[0.86429643	0.13570357]
[0.84968519	0.15031481]
[0.8234341	0.1765659]
[0.84231224	0.15768776]
[0.87095489	0.12904511]

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: Binarizer()

- **Binarizer()**
  - 분류 결정 계수(threshold)에 따른 이진 분류 결과 반환
  - 입력된 threshold 값보다 작으면 0, 크면 1값으로 클래스 분류
  - threshold를 적용하기 위하여 fit()->transform() 또는 fit_transform() 실행: 입력 값은 분류될 대상

```
from sklearn.preprocessing import Binarizer
```

```
from sklearn.preprocessing import Binarizer

X = [[ 1, -1,  2],
     [ 2,  0,  0],
     [ 0,  1.1, 1.2]]

# threshold 기준값보다 같거나 작으면 0을, 크면 1을 반환
binarizer = Binarizer(threshold=1.1)
print(binarizer.fit_transform(X))
```

threshold(1.1) 보다
크면 1, 작으면 0

```
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 0. 1.]]
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 타이타닉 데이터 이진 분류 평가

- **분류 결정 계수**

```python
from sklearn.preprocessing import Binarizer

#Binarizer의 threshold 설정값. 분류 결정 임곗값임.
custom_threshold = 0.5

# predict_proba( ) 반환값의 두번째 컬럼 , 즉 Positive
# 클래스 컬럼 하나만 추출하여 Binarizer를 적용
pred_proba_1 = pred_proba[:,1].reshape(-1,1)

binarizer = Binarizer(threshold = custom_threshold).fit(pred_proba_1)
custom_predict = binarizer.transform(pred_proba_1)

confusion = confusion_matrix( y_test, custom_predict)
accuracy = accuracy_score(y_test , custom_predict)
precision = precision_score(y_test , custom_predict)
recall = recall_score(y_test , custom_predict)
print('오차 행렬')
print(confusion)
print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}'.format(accuracy , precision ,recall))
```

pred_proba

0	1
[0.44935225	0.55064775]
[0.86335511	0.13664489]
[0.86429643	0.13570357]
[0.84968519	0.15031481]
[0.8234341	0.1765659]
[0.84231224	0.15768776]
[0.87095489	0.12904511]

pred_proba_1

실행결과

```
오차 행렬
[[108  10]
 [ 14  47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 타이타닉 데이터 이진 분류 평가

- **분류 결정 계수**
  - custom_threshold값을 변화: 0.4, 0.45, 0.5, 0.55, 0.6

```
# 테스트를 수행할 모든 임곗값을 리스트 객체로 저장.
thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]
pred_proba_c1 = pred_proba[:,1].reshape(-1,1)

# thresholds list객체내의 값을 차례로 iteration하면서 Evaluation 수행.
for custom_threshold in thresholds:
    binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_c1)
    custom_predict = binarizer.transform(pred_proba_c1)
    print('임곗값:',custom_threshold)

    confusion = confusion_matrix( y_test, custom_predict)
    accuracy = accuracy_score(y_test , custom_predict)
    precision = precision_score(y_test , custom_predict)
    recall = recall_score(y_test , custom_predict)
    print('오차 행렬')
    print(confusion)
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}'.format(accuracy , precision ,recall))
    print()
```

0.4, 0.45, 0.5, 0.55, 0.6

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 타이타닉 데이터 이진 분류 평가

- **분류 결정 계수**
  - custom_threshold값을 변화: 0.4, 0.45, 0.5, 0.55, 0.6

```
임곗값: 0.4
오차 행렬
[[97 21]
 [11 50]]
정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197

임곗값: 0.45
오차 행렬
[[105  13]
 [ 13  48]]
정확도: 0.8547, 정밀도: 0.7869, 재현율: 0.7869

임곗값: 0.5
오차 행렬
[[108  10]
 [ 14  47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
```

```
임곗값: 0.55
오차 행렬
[[111   7]
 [ 16  45]]
정확도: 0.8715, 정밀도: 0.8654, 재현율: 0.7377

임곗값: 0.6
오차 행렬
[[113   5]
 [ 17  44]]
정확도: 0.8771, 정밀도: 0.8980, 재현율: 0.7213
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 타이타닉 데이터 이진 분류 평가

- **precision_recall_curve()**
  - 임곗값에 따른 정밀도, 재현율 변화값을 제공
  - 입력 값: 검증 타겟 레이블, 예측된 결과의 분류 확률

```
from sklearn.metrics import precision_recall_curve
```

```
from sklearn.metrics import precision_recall_curve

# 레이블 값이 1일때의 예측 확률을 추출
pred_proba_class1 = lr_clf.predict_proba(X_test)[:, 1]

# 실제값 데이터 셋과 레이블 값이 1일 때의 예측 확률을 precision_recall_curve 인자로 입력
precisions, recalls, thresholds = precision_recall_curve(y_test, pred_proba_class1 )

print('반환된 분류 결정 임곗값 배열의 Shape:', thresholds.shape)
print('반환된 precisions 배열의 Shape:', precisions.shape)
print('반환된 recalls 배열의 Shape:', recalls.shape)

print("thresholds 5 sample:", thresholds[:5])
print("precisions 5 sample:", precisions[:5])
print("recalls 5 sample:", recalls[:5])
```

pred_proba_class1

0	1
[0.44935225	0.55064775]
[0.86335511	0.13664489]
[0.86429643	0.13570357]
[0.84968519	0.15031481]
[0.8234341	0.1765659]
[0.84231224	0.15768776]
[0.87095489	0.12904511]

pred_proba_1

```
반환된 분류 결정 임곗값 배열의 Shape: (165,)
반환된 precisions 배열의 Shape: (166,)
반환된 recalls 배열의 Shape: (166,)
thresholds 5 sample: [0.01974988 0.06956414 0.08402808 0.08474207 0.08920161]
precisions 5 sample: [0.34078212 0.34269663 0.34463277 0.34659091 0.34857143]
recalls 5 sample: [1. 1. 1. 1. 1.]
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 타이타닉 데이터 이진 분류 평가

- **precision_recall_curve()의 결과 값**

```
#반환된 임계값 배열 로우가 165건이므로 샘플로 10건만 추출하되, 임곗값을 15 Step으로 추출.
thr_index = np.arange(0, thresholds.shape[0], 15)
print('샘플 추출을 위한 임계값 배열의 index 10개:', thr_index)
print('샘플용 10개의 임곗값: ', np.round(thresholds[thr_index], 2))

# 15 step 단위로 추출된 임계값에 따른 정밀도와 재현율 값
print('샘플 임계값별 정밀도: ', np.round(precisions[thr_index], 3))
print('샘플 임계값별 재현율: ', np.round(recalls[thr_index], 3))
```

```
샘플 추출을 위한 임계값 배열의 index 10개: [  0  15  30  45  60  75  90 105 120 135 150]
샘플용 10개의 임곗값:  [0.02 0.11 0.13 0.14 0.16 0.24 0.32 0.45 0.62 0.73 0.87]
샘플 임계값별 정밀도:  [0.341 0.372 0.401 0.44  0.505 0.598 0.688 0.774 0.915 0.968 0.938]
샘플 임계값별 재현율:  [1.    1.    0.967 0.902 0.902 0.902 0.869 0.787 0.705 0.492 0.246]
```

정밀도 / 재현율 트레이드 오프
- Scikit-learn 구현: 타이타닉 데이터 이진 분류 평가

- 임곗값의 변경에 따른 정밀도-재현율 변화 곡선

```
import matplotlib.pyplot as plt
%matplotlib inline

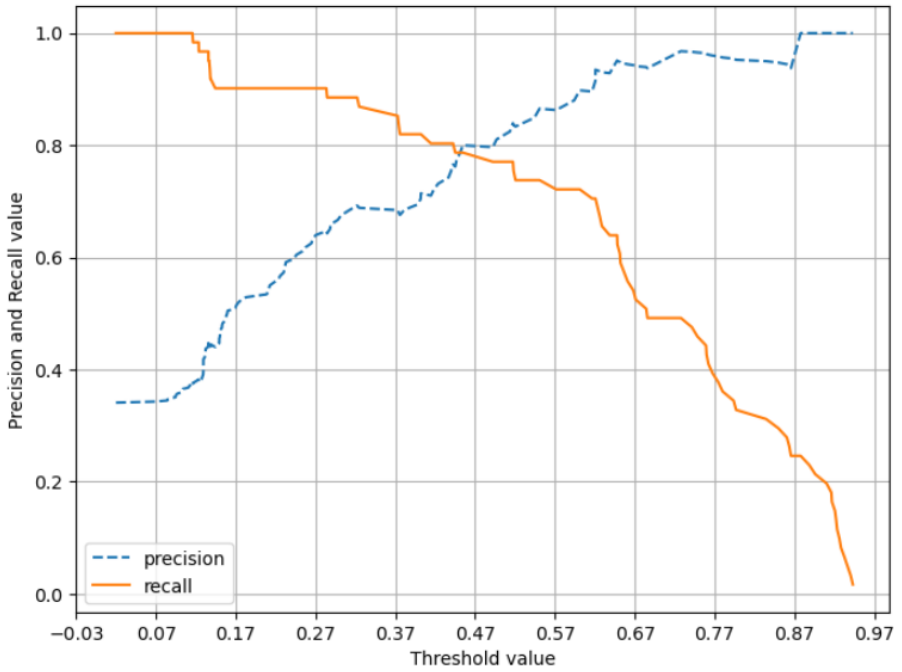
def precision_recall_curve_plot(y_test , pred_proba_c1):
    # threshold ndarray와 이 threshold에 따른 정밀도, 재현율 ndarray 추출.
    precisions, recalls, thresholds = precision_recall_curve( y_test, pred_proba_c1)

    # X축을 threshold값으로, Y축은 정밀도, 재현율 값으로 각각 Plot 수행. 정밀도는 점선으로 표시
    plt.figure(figsize=(8,6))
    threshold_boundary = thresholds.shape[0]
    plt.plot(thresholds, precisions[0:threshold_boundary], linestyle='--', label='precision')
    plt.plot(thresholds, recalls[0:threshold_boundary],label='recall')

    # threshold 값 X 축의 Scale을 0.1 단위로 변경
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1),2))

    # x축, y축 label과 legend, 그리고 grid 설정
    plt.xlabel('Threshold value'); plt.ylabel('Precision and Recall value')
    plt.legend(); plt.grid()
    plt.show()

precision_recall_curve_plot( y_test, lr_clf.predict_proba(X_test)[:, 1] )
```



F1-score

```
from sklearn.metrics import f1_score
```

- **F1-socre**
  - 정밀도와 재현율은 상호 보완적인 평가 지표로 **trade-off 관계**
  - 하나만 높아도 좋은 모델이라고 할 수 없음

  - 정밀도와 재현율을 결합한 조화 평균(Harmonic Mean)
  - 0~1값을 가지며 값이 클수록 모델이 정확하다고 판단
  - 정밀도와 재현율이 모두 높을 때 높은 값을 갖으며, 클래스 불균형이 있는 데이터에서 잘 동작

$$F1-score = \frac{2}{\frac{1}{recall}+\frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

F1-score
- Scikit-learn 구현

- **f1_score()**
  - F1-score 값을 반환
  - 입력값
    - 검증 타겟 레이블
    - 예측된 결과 값

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score , recall_score , confusion_matrix
from sklearn.preprocessing import Binarizer
from sklearn.metrics import f1_score

# 테스트를 수행할 모든 임곗값을 리스트 객체로 저장.
thresholds = [0.4, 0.45, 0.50, 0.55, 0.60]
pred_proba_c1 = pred_proba[:,1].reshape(-1,1)

# thresholds list객체내의 값을 차례로 iteration하면서 Evaluation 수행.
for custom_threshold in thresholds:
    binarizer = Binarizer(threshold=custom_threshold).fit(pred_proba_c1)
    custom_predict = binarizer.transform(pred_proba_c1)
    print('임곗값:',custom_threshold)

    confusion = confusion_matrix( y_test, custom_predict)
    accuracy = accuracy_score(y_test , custom_predict)
    precision = precision_score(y_test , custom_predict)
    recall = recall_score(y_test , custom_predict)
    F1_Score = f1_score(y_test , custom_predict)

    print('오차 행렬')
    print(confusion)
    print('정확도: {0:.4f}, 정밀도: {1:.4f}, 재현율: {2:.4f}'.format(accuracy , precision ,recall))
    print('F1_Score: {0:.4f}'.format(F1_Score))
    print()
```

F1-score
- Scikit-learn 구현

- **f1_score()의 결과 값**

```
임곗값: 0.4
오차 행렬
[[97 21]
 [11 50]]
정확도: 0.8212, 정밀도: 0.7042, 재현율: 0.8197
F1_Score: 0.7576

임곗값: 0.45
오차 행렬
[[105  13]
 [ 13  48]]
정확도: 0.8547, 정밀도: 0.7869, 재현율: 0.7869
F1_Score: 0.7869

임곗값: 0.5
오차 행렬
[[108  10]
 [ 14  47]]
정확도: 0.8659, 정밀도: 0.8246, 재현율: 0.7705
F1_Score: 0.7966
```

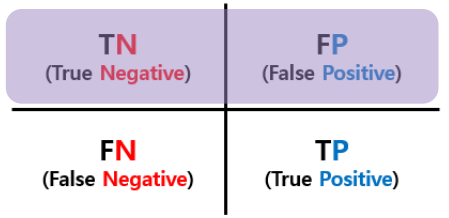
```
임곗값: 0.55
오차 행렬
[[111   7]
 [ 16  45]]
정확도: 0.8715, 정밀도: 0.8654, 재현율: 0.7377
F1_Score: 0.7965

임곗값: 0.6
오차 행렬
[[113   5]
 [ 17  44]]
정확도: 0.8771, 정밀도: 0.8980, 재현율: 0.7213
F1_Score: 0.8000
```

ROC곡선과 AUC

- **특이도(Specificity)**
  - 실제 음성(Negative)인 대상 중에서 음성으로 올바르게 예측한 비율
  - 실제 음성인 값을 양성으로 예측하는 False Positive를 피하는데 초점
    - 특이도 값이 높을수록 음성 데이터를 음성으로 잘 분류(불필요한 양성 오판정이 적음)
    - 특이도 값이 낮을수록 음성 데이터를 음성으로 잘못 분류(거짓 정보가 많음)

$$Specificity = \frac{TN}{TN + FP}$$

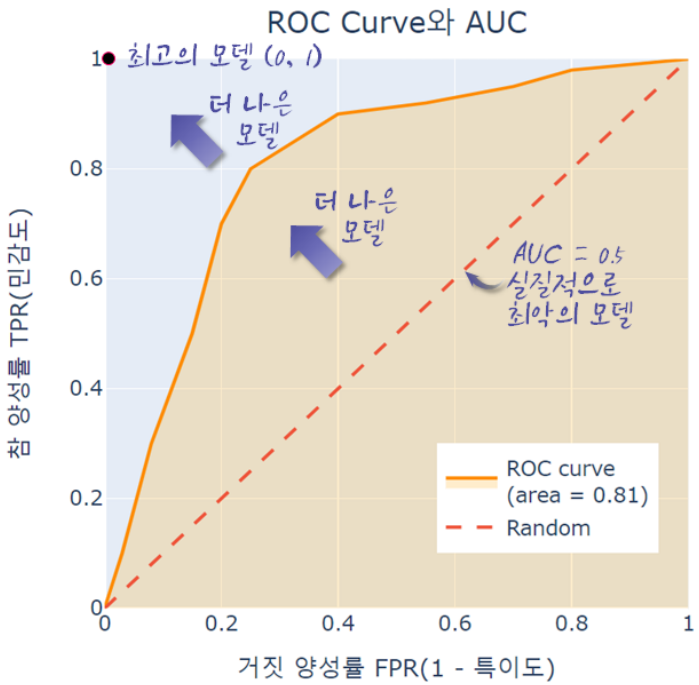


- **FPR(False Positive Rate)**
  - 음성(Negative)인 대상 중에서 양성(Positive)으로 잘못 예측한 비율
  - 음성을 대상으로 예측한 것 중에서 틀린 비율
  - 1-특이도

$$FPR = \frac{FP}{TN + FP} = 1 - Specificity$$

ROC곡선과 AUC

- **ROC 곡선(Receiver Operation Characteristic Curve)**
  - FPR(False Positive Rate)값이 변할 때 TPR(True Positive Rate)값의 변화를 보여주는 곡선
  - 분류의 성능 지표로 사용되며 ROC 곡선 면적에 기반한 AUC 값으로 결정
  - AUC(Area Under Curve)값
    - ROC 곡선 밑의 면적
    - 1에 가까울수록 좋은 수치



ROC곡선과 AUC
- Scikit-learn 구현

- **roc_curve()**
  - 임곗값에 따른 FPR, TPR 변화값을 제공
  - 입력 값: 검증 타겟 레이블, 예측된 결과의 분류 확률

```
from sklearn.metrics import roc_curve
```

```
from sklearn.metrics import roc_curve

# 레이블 값이 1일때의 예측 확률을 추출
pred_proba_class1 = lr_clf.predict_proba(X_test)[:, 1]

fprs , tprs , thresholds = roc_curve(y_test, pred_proba_class1)
# 반환된 임곗값 배열에서 샘플로 데이터를 추출하되, 임곗값을 5 Step으로 추출.
# thresholds[0]은 max(예측확률)+1로 임의 설정됨. 이를 제외하기 위해 np.arange는 1부터 시작

thr_index = np.arange(1, thresholds.shape[0], 5)
print('샘플 추출을 위한 임곗값 배열의 index:', thr_index)
print('샘플 index로 추출한 임곗값: ', np.round(thresholds[thr_index], 2))

# 5 step 단위로 추출된 임계값에 따른 FPR, TPR 값
print('샘플 임곗값별 FPR: ', np.round(fprs[thr_index], 3))
print('샘플 임곗값별 TPR: ', np.round(tprs[thr_index], 3))
```

```
샘플 추출을 위한 임곗값 배열의 index: [ 1  6 11 16 21 26 31 36 41 46]
샘플 index로 추출한 임곗값:  [0.94 0.73 0.62 0.52 0.44 0.28 0.15 0.14 0.13 0.12]
샘플 임곗값별 FPR:  [0.    0.008 0.025 0.076 0.127 0.254 0.576 0.61  0.746 0.847]
샘플 임곗값별 TPR:  [0.016 0.492 0.705 0.738 0.803 0.885 0.902 0.951 0.967 1.   ]
```

ROC곡선과 AUC
- Scikit-learn 구현

- **ROC 곡선 및 AUC**

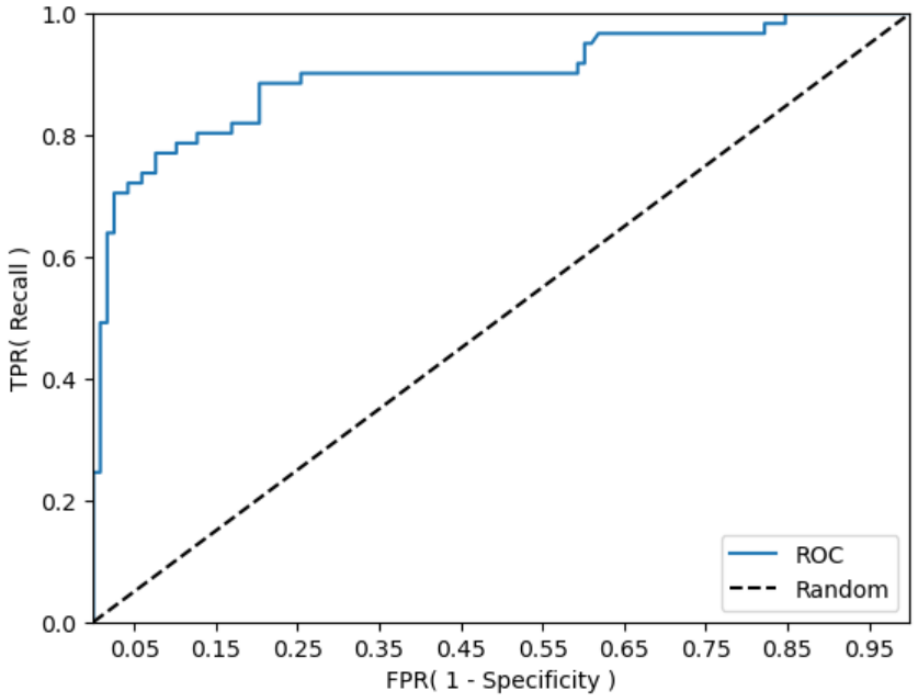
```
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt
%matplotlib inline

def roc_curve_plot(y_test , pred_proba_c1):
    # 임곗값에 따른 FPR, TPR 값을 반환 받음.
    fprs , tprs , thresholds = roc_curve(y_test ,pred_proba_c1)

    # ROC Curve를 plot 곡선으로 그림.
    plt.plot(fprs , tprs, label='ROC')
    # 가운데 대각선 직선을 그림.
    plt.plot([0, 1], [0, 1], 'k--', label='Random')

    # FPR X 축의 Scale을 0.1 단위로 변경, X,Y 축명 설정등
    start, end = plt.xlim()
    plt.xticks(np.round(np.arange(start, end, 0.1),2))
    plt.xlim(0,1); plt.ylim(0,1)
    plt.xlabel('FPR( 1 - Specificity )'); plt.ylabel('TPR( Recall )')
    plt.legend()
    plt.show()

roc_curve_plot(y_test, lr_clf.predict_proba(X_test)[:, 1] )
```



ROC곡선과 AUC
- Scikit-learn 구현

- **roc_auc_score()**
  - ROC곡선에 의한 AUC의 면적을 반환
  - 1에 가까울 수록 좋은 성능으로 평가
  - 입력 값: 검증 타겟 레이블, 예측된 결과의 분류 확률

```
from sklearn.metrics import roc_auc_score
```

```
from sklearn.metrics import roc_auc_score

pred_proba = lr_clf.predict_proba(X_test)[:, 1]
roc_score = roc_auc_score(y_test, pred_proba)
print('ROC AUC 값: {0:.4f}'.format(roc_score))
```

```
ROC AUC 값: 0.8987
```



강남대학교

KANGNAM UNIVERSITY